UNIVERSIDAD TÉCNICA FEDERICO SANTA MARÍA

Departamento de Industrias · Campus Vitacura

LABORATORIO 5

Entregable SQL (2/2) · Andes Bike

ICN-292 · Sistemas de Información para la Gestión · 2026/02

Estudiante: Matías Castro

Profesores: José Miguel González Paul · José Luis Sáez Tamayo · Patricio Cifuentes

Repositorio GitHub (HTTPS): https://github.com/matycastroc1/ICN292-lab5-Matias-Castro

Fecha: 10 de noviembre de 2026

ENTREGABLE 1 — CREACIÓN Y CARGA DE LA BASE DE DATOS

Base de datos: andesbike

1. Creación de la base de datos y las cuatro tablas

```
CREATE DATABASE IF NOT EXISTS andesbike;
USE andesbike;

CREATE TABLE tiendas (
 id_tienda INT AUTO_INCREMENT PRIMARY KEY,
 nombre VARCHAR(60) NOT NULL,
 comuna VARCHAR(50) NOT NULL
);
CREATE TABLE vendedores (
 id_vendedor INT AUTO_INCREMENT PRIMARY KEY,
 nombre VARCHAR(80) NOT NULL,
 id_tienda INT NULL,
 fecha_ingreso DATE NOT NULL,
 FOREIGN KEY (id_tienda) REFERENCES tiendas(id_tienda)
);
CREATE TABLE bicicletas (
 id_bicicleta INT AUTO_INCREMENT PRIMARY KEY,
 modelo VARCHAR(60) NOT NULL,
 categoria ENUM('Eléctrica','Infantil','Montaña','Ruta','Urbana') NOT NULL,
 precio DECIMAL(10,2) NOT NULL,
 CHECK (precio >= 0)
);
CREATE TABLE ventas (
 id_venta INT AUTO_INCREMENT PRIMARY KEY,
 id_vendedor INT NOT NULL,
 id_bicicleta INT NOT NULL,
 fecha DATE NOT NULL,
 unidades INT NOT NULL,
 CHECK (unidades > 0),
 FOREIGN KEY (id_vendedor) REFERENCES vendedores(id_vendedor),
 FOREIGN KEY (id_bicicleta) REFERENCES bicicletas(id_bicicleta)
);
```

2. Carga oficial de los registros

```
INSERT INTO tiendas (nombre, comuna) VALUES
('Andes Providencia','Providencia'),('Andes Maipú','Maipú'),
('Andes Concepción','Concepción'),('Andes La Serena','La Serena');
INSERT INTO vendedores (nombre, id_tienda, fecha_ingreso) VALUES
('Camila Ortiz',1,'2024-06-20'),('Diego Fuentes',1,'2024-01-15'),
('Javiera Soto',2,'2023-07-24'),('Matías Herrera',2,'2024-01-15'),
('Paula Riquelme',3,'2022-11-02'),('Ignacio Vera',3,'2024-08-19'),
('Fernanda Lillo',1,'2025-02-03'),('Rubén Castro',NULL,'2025-06-10');
INSERT INTO bicicletas (modelo, categoria, precio) VALUES
('Cordillera 29','Montaña',549990.00),('Ruta Austral','Ruta',899990.00),
('Urbana Pro','Urbana',329990.00),('Pequeño Andes','Infantil',149990.00),
('Volt City','Eléctrica',1249990.00),('Trail Master','Montaña',749990.00),
('Urbana Lite','Urbana',229990.00),('Sprint R7','Ruta',1099990.00);
INSERT INTO ventas (id_vendedor, id_bicicleta, fecha, unidades) VALUES
(1,1,'2025-03-04',2),(1,3,'2025-03-18',1),(1,5,'2025-04-02',1),
(2,2,'2025-03-04',1),(2,1,'2025-04-11',3),(2,7,'2025-05-06',2),
(3,4,'2025-03-21',4),(3,3,'2025-04-15',2),(3,6,'2025-05-20',1),
(4,1,'2025-03-27',1),(4,5,'2025-05-09',2),(5,2,'2025-04-08',1),
(5,6,'2025-04-23',2),(5,4,'2025-06-02',5),(6,3,'2025-05-13',3),
(6,7,'2025-06-17',1),(6,1,'2025-06-24',2),(1,6,'2025-06-30',1);
```

3. Comprobación de la carga

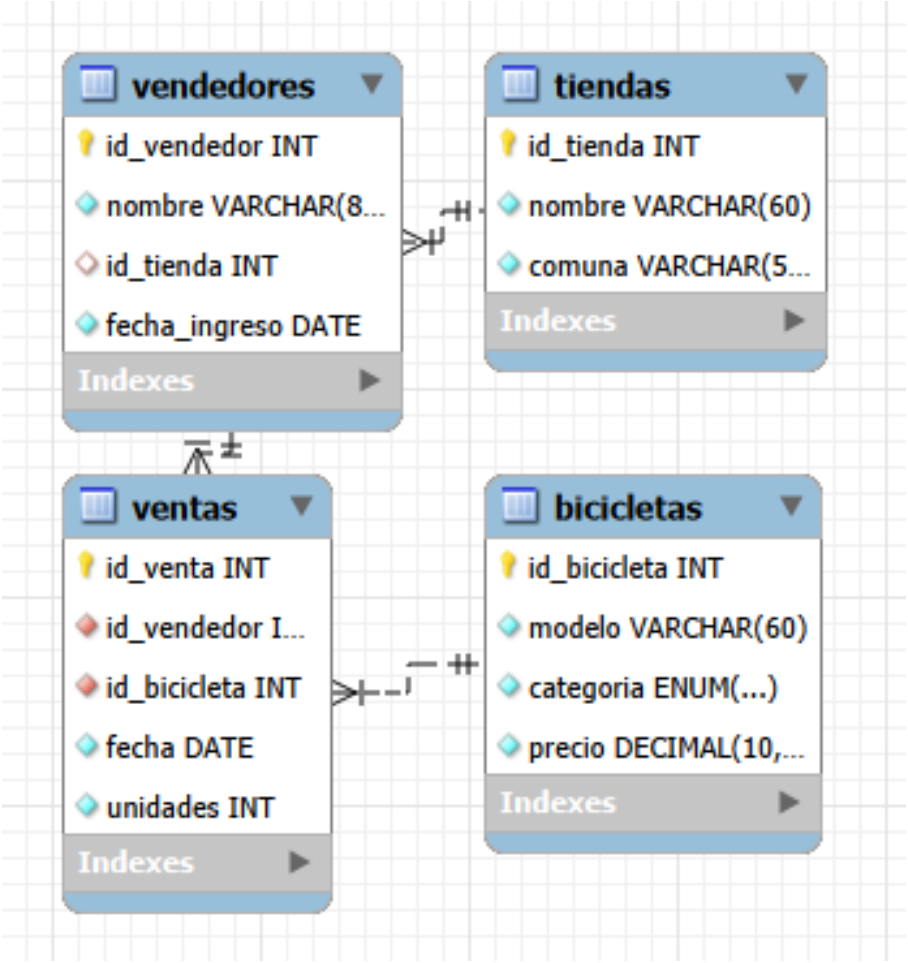
Carga oficial: 4 tiendas, 8 vendedores, 8 modelos de bicicletas y 18 ventas. Se comprobó un total de 35 unidades vendidas y un monto de $16.969.650.

```
SELECT COUNT(*) AS total_ventas, SUM(unidades) AS total_unidades
FROM ventas;


SELECT SUM(v.unidades * b.precio) AS monto_total
FROM ventas v
INNER JOIN bicicletas b ON v.id_bicicleta = b.id_bicicleta;
```

4. Modelo entidad–relación

Diagrama obtenido mediante Database > Reverse Engineer en MySQL Workbench. Contiene las tablas tiendas, vendedores, bicicletas y ventas, con sus tres relaciones.



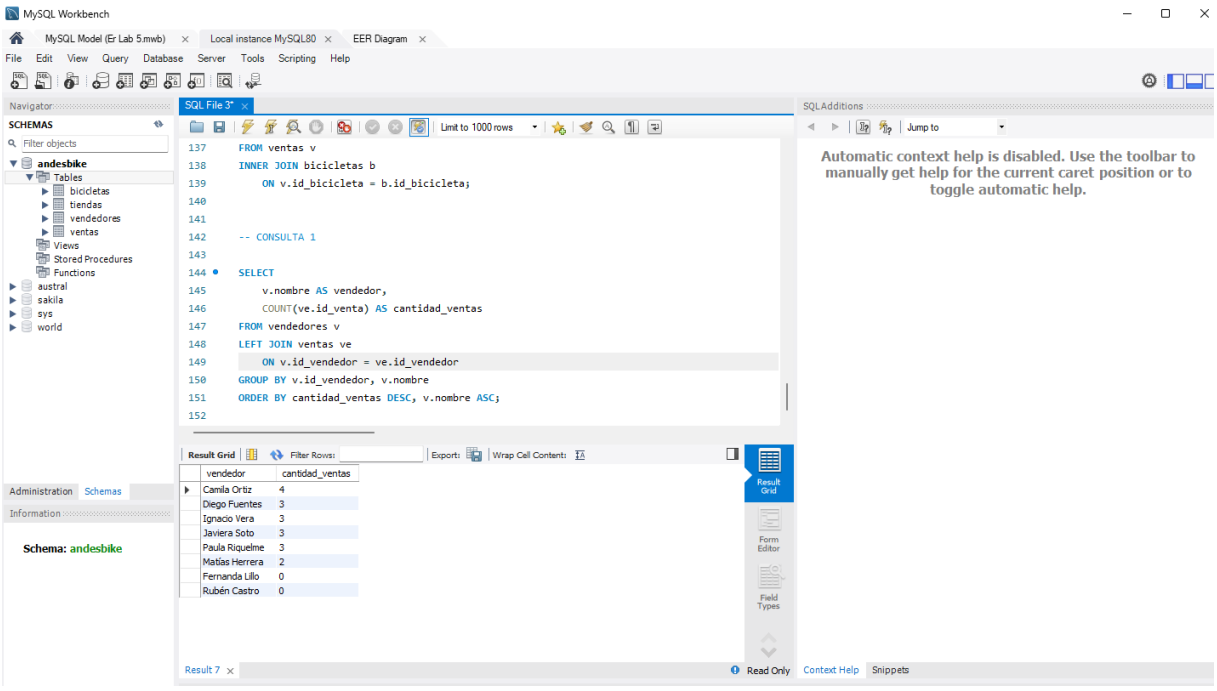
ENTREGABLE 2 — CONSULTAS SQL

Consulta 1. Ventas registradas por vendedor

Código SQL

```
SELECT
   v.nombre AS vendedor,
   COUNT(ve.id_venta) AS cantidad_ventas
FROM vendedores v
LEFT JOIN ventas ve
   ON v.id_vendedor = ve.id_vendedor
GROUP BY v.id_vendedor, v.nombre
ORDER BY cantidad_ventas DESC, v.nombre ASC;
```

Resultado en MySQL Workbench

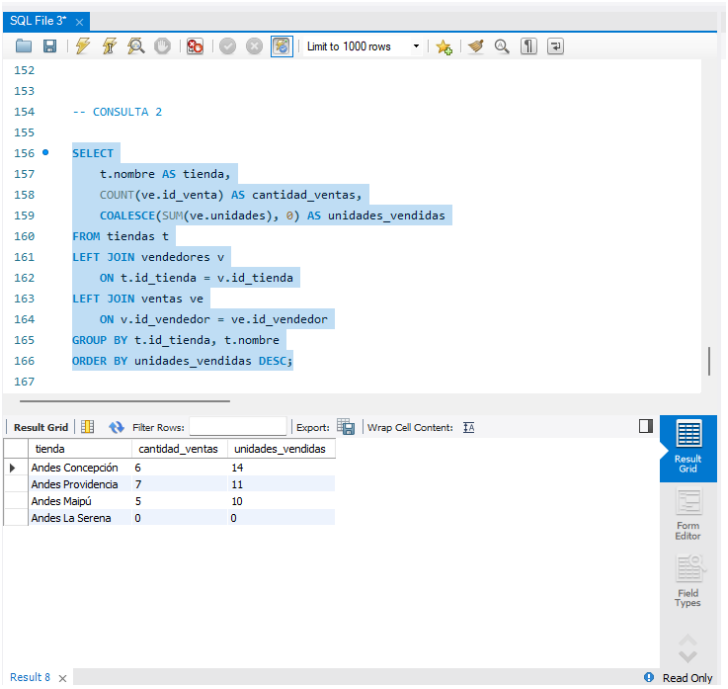


Consulta 2. Ventas y unidades por tienda

Código SQL

```
SELECT
   t.nombre AS tienda,
   COUNT(ve.id_venta) AS cantidad_ventas,
   COALESCE(SUM(ve.unidades), 0) AS unidades_vendidas
FROM tiendas t
LEFT JOIN vendedores v
   ON t.id_tienda = v.id_tienda
LEFT JOIN ventas ve
   ON v.id_vendedor = ve.id_vendedor
GROUP BY t.id_tienda, t.nombre
ORDER BY unidades_vendidas DESC;
```

Resultado en MySQL Workbench

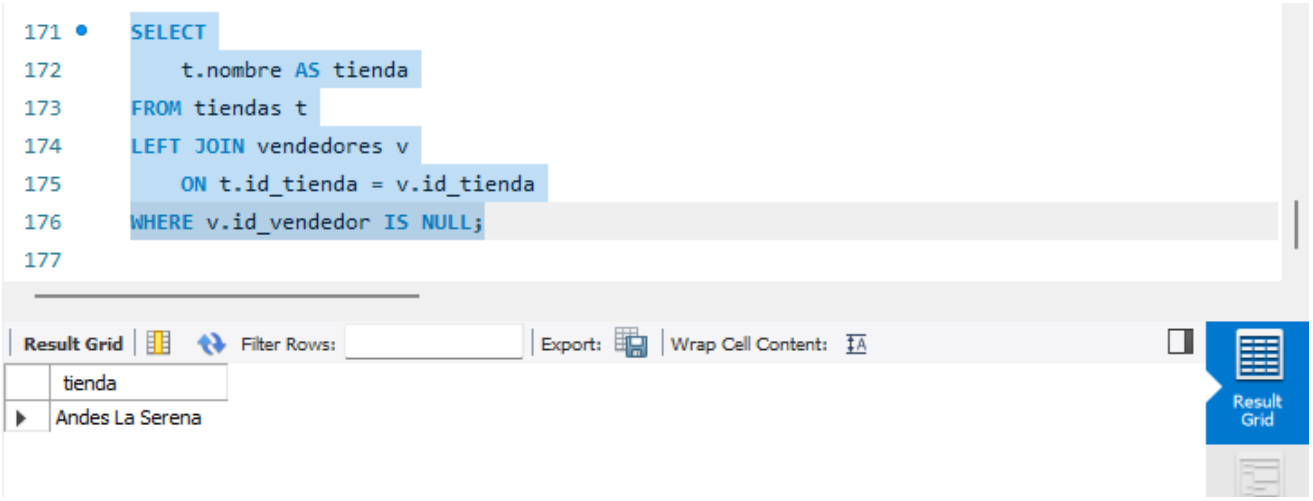


Consulta 3. Tiendas sin vendedores

Código SQL

```
SELECT
   t.nombre AS tienda
FROM tiendas t
LEFT JOIN vendedores v
   ON t.id_tienda = v.id_tienda
WHERE v.id_vendedor IS NULL;
```

Resultado en MySQL Workbench

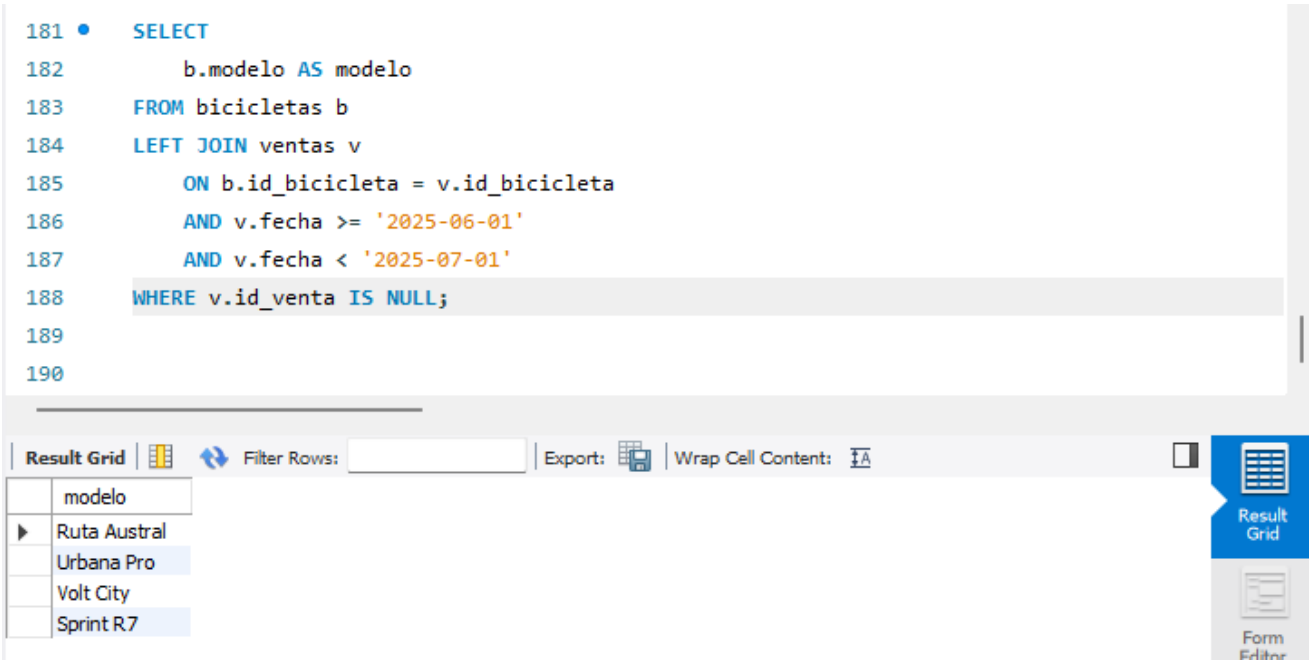


Consulta 4. Modelos sin ventas en junio de 2025

Código SQL

```
SELECT
   b.modelo AS modelo
FROM bicicletas b
LEFT JOIN ventas v
   ON b.id_bicicleta = v.id_bicicleta
   AND v.fecha >= '2025-06-01'
   AND v.fecha < '2025-07-01'
WHERE v.id_venta IS NULL;
```

Resultado en MySQL Workbench



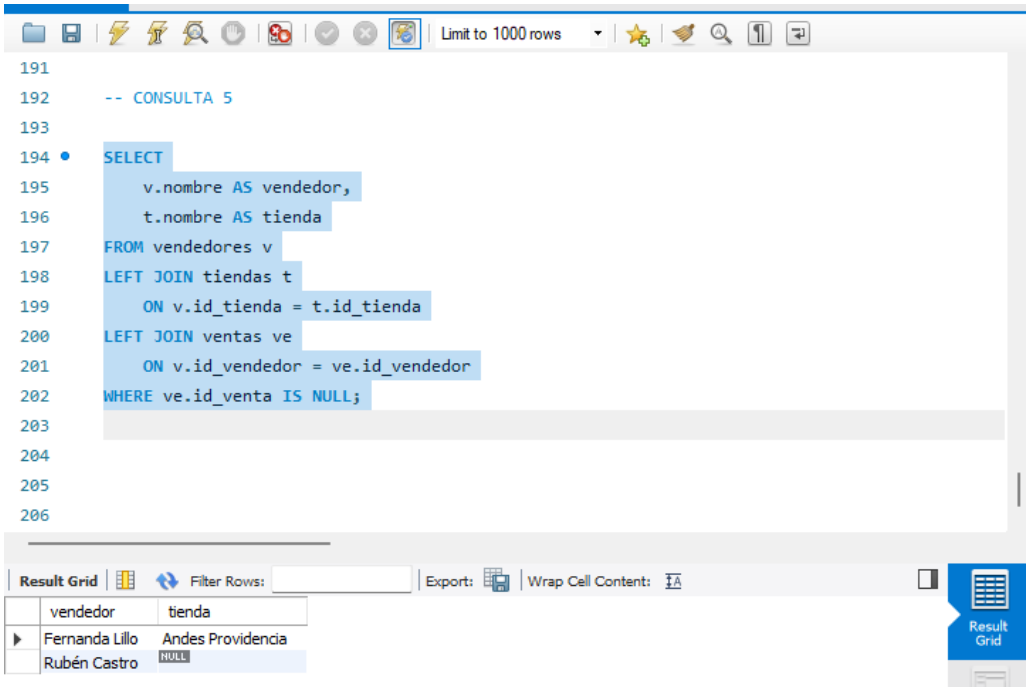
Explicación solicitada: La condición de junio se colocó en el ON del LEFT JOIN; si se trasladara al WHERE conservando el filtro IS NULL, no se obtendrían los modelos sin ventas en ese mes.

Consulta 5. Vendedores sin ventas

Código SQL

```
SELECT
   v.nombre AS vendedor,
   t.nombre AS tienda
FROM vendedores v
LEFT JOIN tiendas t
   ON v.id_tienda = t.id_tienda
LEFT JOIN ventas ve
   ON v.id_vendedor = ve.id_vendedor
WHERE ve.id_venta IS NULL;
```

Resultado en MySQL Workbench

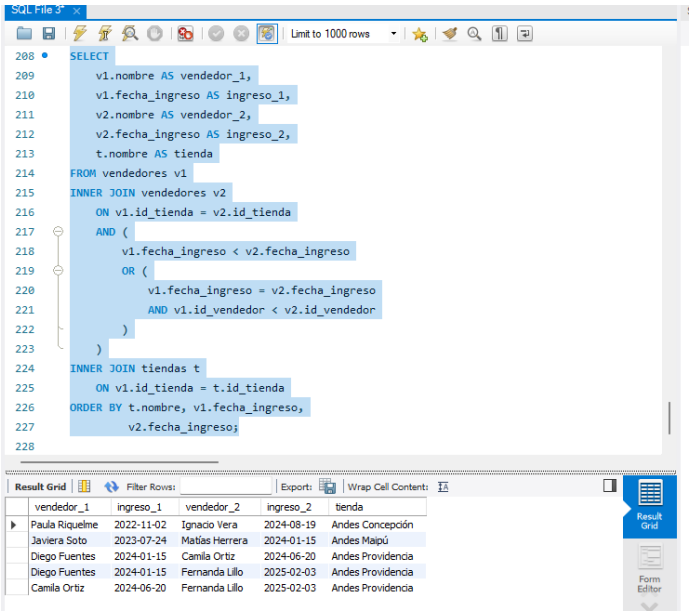


Consulta 6. Pares de vendedores de una misma tienda

Código SQL

```
SELECT
   v1.nombre AS vendedor_1,
   v1.fecha_ingreso AS ingreso_1,
   v2.nombre AS vendedor_2,
   v2.fecha_ingreso AS ingreso_2,
   t.nombre AS tienda
FROM vendedores v1
INNER JOIN vendedores v2
   ON v1.id_tienda = v2.id_tienda
   AND (
      v1.fecha_ingreso < v2.fecha_ingreso
      OR (
         v1.fecha_ingreso = v2.fecha_ingreso
         AND v1.id_vendedor < v2.id_vendedor
      )
   )
INNER JOIN tiendas t
   ON v1.id_tienda = t.id_tienda
ORDER BY t.nombre, v1.fecha_ingreso,
       v2.fecha_ingreso;
```

Resultado en MySQL Workbench

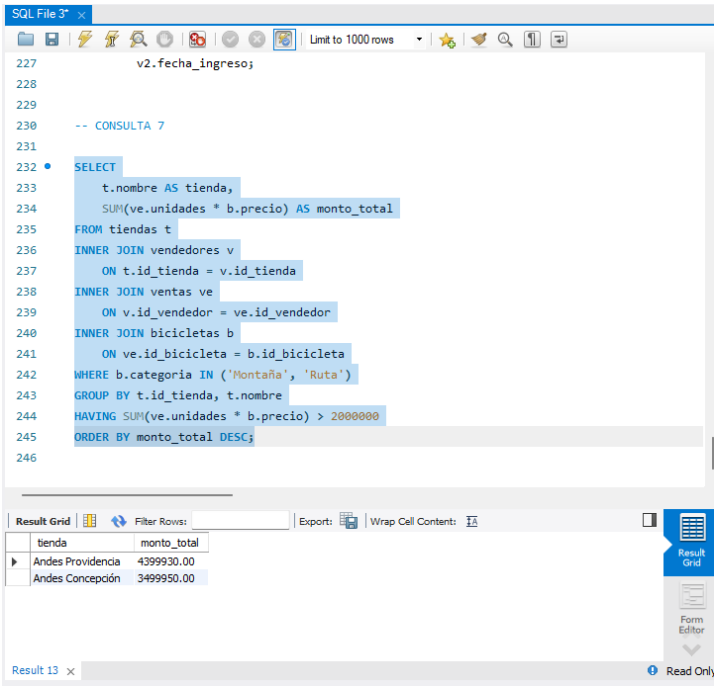


Consulta 7. Monto por tienda: Montaña y Ruta

Código SQL

```
SELECT
   t.nombre AS tienda,
   SUM(ve.unidades * b.precio) AS monto_total
FROM tiendas t
INNER JOIN vendedores v
   ON t.id_tienda = v.id_tienda
INNER JOIN ventas ve
   ON v.id_vendedor = ve.id_vendedor
INNER JOIN bicicletas b
   ON ve.id_bicicleta = b.id_bicicleta
WHERE b.categoria IN ('Montaña', 'Ruta')
GROUP BY t.id_tienda, t.nombre
HAVING SUM(ve.unidades * b.precio) > 2000000
ORDER BY monto_total DESC;
```

Resultado en MySQL Workbench

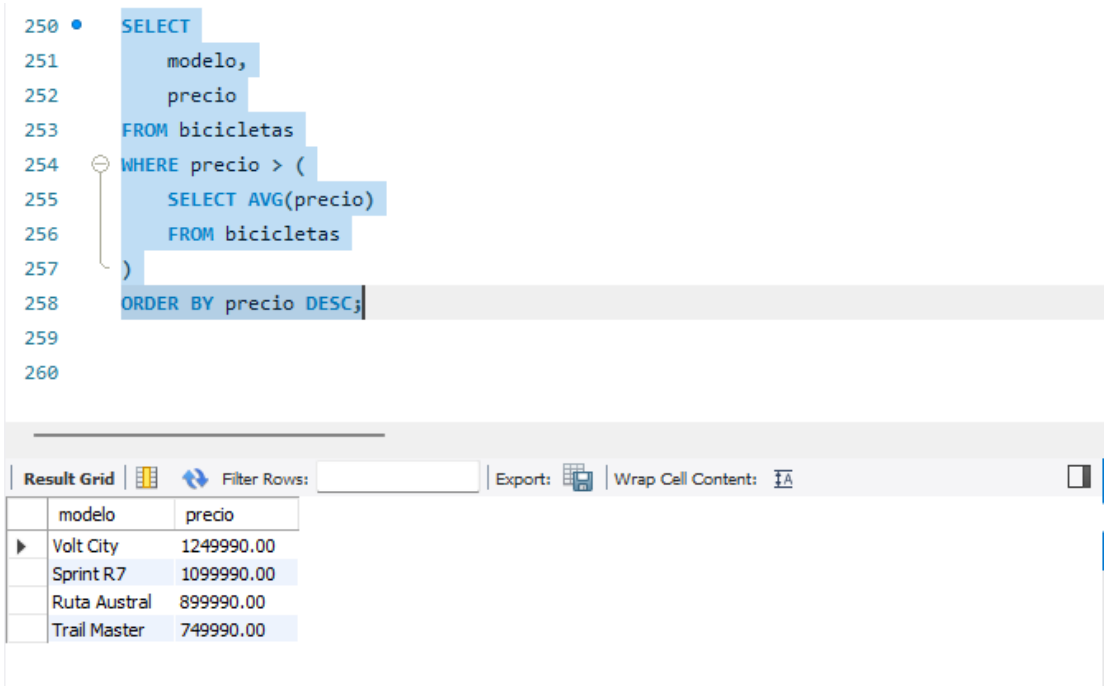


Consulta 8. Bicicletas de precio superior al promedio

Código SQL

```
SELECT
   modelo,
   precio
FROM bicicletas
WHERE precio > (
   SELECT AVG(precio)
   FROM bicicletas
)
ORDER BY precio DESC;
```

Resultado en MySQL Workbench



Explicación solicitada: El precio promedio es $657.490, obtenido con AVG(precio) sobre los ocho registros de la tabla bicicletas.